
CSE 234 Project 1 Report

Ewan Shen

Department of Computer Science
University of California, San Diego
ewshen@ucsd.edu

Shinjini Halder

Department of Computer Science
University of California, San Diego
shhalder@ucsd.edu

1 Data Creation Methodology

I designed the QA pairs through semi-synthetic generation. Inspired by the lecture on bootstrapping evaluation datasets, we implemented a two-step process to generate random combinations of dimensions and queries based on realistic scenarios and constraints.

For the first step, we defined the dimensions, including document file, question type, and scenario. The document file is the relevant file. The question type defines the genre of the user query, including factual lookup, comparison, procedural facts, or conceptual explanations, to ensure question diversity that is representative of how a real-world user would ask varied questions. Lastly, the scenario dimension describes the reasoning method to resolve the query, such as whether the answer is an exact lookup directly present in the corpus, or a synthesis of various texts or files, or an edge case that defines system limits. We also added a diversity constraint to the prompt to ensure that at least 15 unique documents have queries. We also required the prompt to generate no more than 2 repeats of any document and 5 questions per question type to ensure a balanced distribution of questions. We also constrained the system to avoid duplicate tuples.

For the second step, we defined question constraints before query generation, such as the requirement for the answer to be concretely found within the documents within line spans, the requirement for the question to be specific and nontrivial, the requirement for no close duplicates, and finally the preference for single file context with at most 4 questions drawing from multiple files. We concluded our prompt with a JSON formatting constraint and an instruction to check and enforce constraints, requiring regeneration until all rules are met. After generating our diverse queries using the two-step process, we synthetically generated answers with LLMs, providing concise reference answers with line spans. We used a human-in-the-loop process to verify that the synthesized queries are diverse, representative of realistic questions, and conform to constraints. For each question, we manually verified the reference answer in the corpus and the associated line spans.

Regarding iteration, we initially observed question paraphrases before implementing our balanced distribution constraint of generating with at least 15 unique documents. While the questions were different, many queries were from the same documents, decreasing the diversity of the evaluations. After adding our constraint, queries were generated for a balanced distribution of documents. Furthermore, during human-in-the-loop review, we removed and regenerated awkward queries that we felt did not represent realistic questions.

Regarding additional LLM metrics, we further ensured question specificity and diversity through LLM verification. While we only briefly fed the 20 queries to an LLM to ensure that all constraints were met, we wanted an additional verification in case of human error. As a result, we semi-synthesized 20 golden QA evaluation queries.

2 Final Pipeline Architecture

Our final RAG pipeline begins with chunking. The RapidFire AI corpus exists as “.rst” files. Regarding the chunking strategy, we used LangChain’s RecursiveCharacterTextSplitter function to split text into chunks. The LangChain function attempts to first chunk text by natural boundaries,

such as paragraph breaks. After the experiment, we also decided on a chunk size of 288, which means each chunk will be a maximum of 288 tokens. We also decided on a chunk overlap of 72 tokens, which means neighboring chunks share 72 tokens to utilize the surrounding semantic context. We also included an `add_start_index` flag to track where in which documents the chunks originate from. Next, we embed the vectors with the BAAI/bge-small-en-v1.5 encoder with a batch size of 32, so 32 chunks are embedded into vectors at once. We then store the vectors in the FAISS vector database.

For retrieval, we use MMR (Max Marginal Relevance) over strict similarity-based searches to improve retrieved chunk diversity. We also decided on the top 40 relevant chunks being retrieved after experimentation and evaluation. We then rerank the top 40 chunks with a cross encoder (ms-marco-MiniLM-L6-v2) to rank the most relevant remaining chunks. The cross encoder takes both the query and the chunk and assigns a relevancy score, which is used for reranking. We then keep the top 8 reranked chunks to further guide context. As the chunks are added in order, we abide by the 2000 token budget limit by preventing chunks from being added when they would hit a limit of 1600 tokens overall. We reserve 400 tokens for the system prompt and user query.

For prompt design, we include a system instruction prompting the LLM to act in the role of a technical document assistant that provides grounded answers to queries. We also include a few constraints for the system to strictly use the provided context and to refrain from answering when the context does not provide the answer to the query, to prevent hallucinations. Finally, the prompt is stitched together from the system instruction, the user query, and the retrieved context.

3 Experimentation Methodology

Table 1: RapidFire AI Multi-Configuration Experiments

Run	Chunk	Overlap	k	Embed Model	Search	Reranker	HyDE	Fusion	Retrieval	F1
1	288	72	40	BGE-small-v1.5	MMR	MiniLM-L6-v2	False	False	0.635	0.526
2	288	72	40	BGE-large-v1.5	MMR	MiniLM-L6-v2	False	False	0.615	0.495
3	288	72	40	GTE-large	MMR	MiniLM-L6-v2	False	False	0.630	0.519
4	288	72	40	BGE-large-v1.5	MMR	BGE-reranker-base	False	False	0.566	0.467
5	288	72	40	BGE-large-v1.5	MMR	MiniLM-L6-v2	True	False	0.622	0.507
6	288	72	40	BGE-large-v1.5	MMR	BGE-reranker-base	False	True	0.603	0.479
7	288	72	40	GTE-large	MMR	BGE-reranker-base	False	False	0.605	0.504
8	256	64	40	BGE-small-v1.5	MMR	MiniLM-L6-v2	False	False	0.570	0.500
9	224	56	40	BGE-small-v1.5	MMR	MiniLM-L6-v2	False	False	0.560	0.490
10	288	72	40	BGE-small-v1.5	Similarity	MiniLM-L6-v2	False	False	0.570	0.500

These are the experiments we have run on RapidFire AI. The first evaluation shows a retrieval score of approximately **0.57** with `chunk_size` = 256 and `chunk_overlap` = 64. Initial experimentation focused on decreasing chunk size because smaller chunks can improve retrieval granularity and reduce irrelevant context, and decreasing overlap because lower overlap reduces redundancy between neighboring chunks and improves throughput. This slightly changed the score, but retrieval quality did not improve significantly.

We then stuck to the best-performing baseline results around (256, 64) and moved with more experiments on both smaller and larger chunking configurations. We experimented with (224, 56) and later (288, 72). The larger chunking configuration (288, 72) achieved the best retrieval score of **0.6353**, while maintaining similar throughput and zero judge failures. This suggests that larger contextual chunks preserved semantic continuity better and improved retrieval effectiveness.

Since embeddings are responsible for converting textual chunks into semantic vector representations that determine retrieval similarity, we also experimented with different embedding models to observe their effect on retrieval quality. We evaluated embedding models such as BAAI/bge-small-en-v1.5, BAAI/bge-large-en-v1.5, and thenlper/gte-large. Different embedding models capture semantic meaning differently, which directly impacts how relevant documents are retrieved from the vector database. From the results, BAAI/bge-small-en-v1.5 produced the best retrieval score of **0.635**, while BAAI/bge-large-en-v1.5 and thenlper/gte-large achieved retrieval scores around **0.615–0.630**. This demonstrates that embedding selection plays a major role in RAG quality and that larger embedding models do not always guarantee better retrieval performance.

We also experimented with reranking. Reranking helps by reordering initially retrieved documents using a stronger relevance model after the vector search stage. In our project, reranking helps improve the final context quality sent to the LLM by prioritizing the most semantically relevant chunks and filtering weaker matches from the top-k retrieved results. We evaluated rerankers such as `cross-encoder/ms-marco-MiniLM-L6-v2` and `BAAI/bge-reranker-base`. From the results, we observed that reranking improved retrieval consistency and reduced noisy context being passed into generation. However, stronger rerankers occasionally introduced additional latency and slightly reduced throughput due to the extra stage.

We additionally experimented with advanced retrieval strategies such as HyDE (Hypothetical Document Embeddings) and score fusion. HyDE generates hypothetical answers before retrieval to improve semantic search quality, while score fusion combines retrieval and reranker scores to improve ranking robustness. The HyDE configuration achieved a retrieval score of **0.622**, showing improvement over several baseline configurations, while score fusion experiments demonstrated balanced retrieval quality and ranking stability.

We also tried multiple models such as:

- `api-mistral-small-3.2-2506`: a lightweight and efficient instruction-tuned model used for fast experimentation and retrieval evaluation because it provides a strong balance between reasoning capability and inference speed.
- `BAAI/bge-small-en-v1.5`: used as the primary embedding model because it achieved the best retrieval quality while maintaining efficient inference speed.
- `BAAI/bge-large-en-v1.5`: a larger embedding model evaluated to test whether larger semantic representations improve retrieval performance.
- `thenlper/gte-large`: another high-capacity embedding model evaluated for semantic retrieval robustness.
- `cross-encoder/ms-marco-MiniLM-L6-v2`: a lightweight reranker used to improve relevance ordering of retrieved chunks.
- `BAAI/bge-reranker-base`: a stronger reranker model evaluated to improve retrieval ranking quality further.

Additionally, we used the multi-configuration experimentation API provided by RapidFire AI, specifically `RFGGridSearch`, to automate large-scale experimentation across chunk sizes, overlaps, embedding models, rerankers, HyDE configurations, and score fusion settings. This allowed us to systematically evaluate many parameter combinations efficiently and compare retrieval metrics such as retrieval score, F1 score, precision, recall, generation quality, throughput, and judge failures across runs. Through `RFGGridSearch`, we identified that the (288, 72) chunking configuration combined with `BAAI/bge-small-en-v1.5`, MMR retrieval, and `cross-encoder/ms-marco-MiniLM-L6-v2` reranking consistently produced the best retrieval performance for our dataset and query distribution.

4 Results and Analysis

We evaluated multiple RAG pipeline configurations using RapidFire AI and systematically analyzed how retrieval parameters, embedding models, rerankers, and retrieval strategies affected both retrieval quality and generation quality. Our initial baseline configuration used `chunk_size = 256`, `chunk_overlap = 64`, `k = 40`, MMR retrieval, and `BAAI/bge-small-en-v1.5` embeddings, which achieved a retrieval score of approximately **0.57**. The initial motivation behind reducing chunk size was that smaller chunks generally improve retrieval granularity and reduce irrelevant context being retrieved. Similarly, lower overlap values reduce redundancy between neighboring chunks and improve retrieval efficiency and throughput.

We then experimented with smaller chunking configurations such as (224, 56). While this slightly improved throughput, retrieval quality decreased because the smaller chunks fragmented semantic context too aggressively. Important contextual relationships between neighboring text segments were lost, making retrieval less semantically complete. After observing this behavior, we moved toward larger chunking configurations and evaluated (288, 72). This configuration achieved the best retrieval score of approximately **0.635**, significantly outperforming the smaller chunk configurations

while maintaining nearly identical throughput and zero judge failures. This result suggests that larger contextual windows preserved semantic continuity more effectively and helped retrieve more complete supporting evidence for generation.

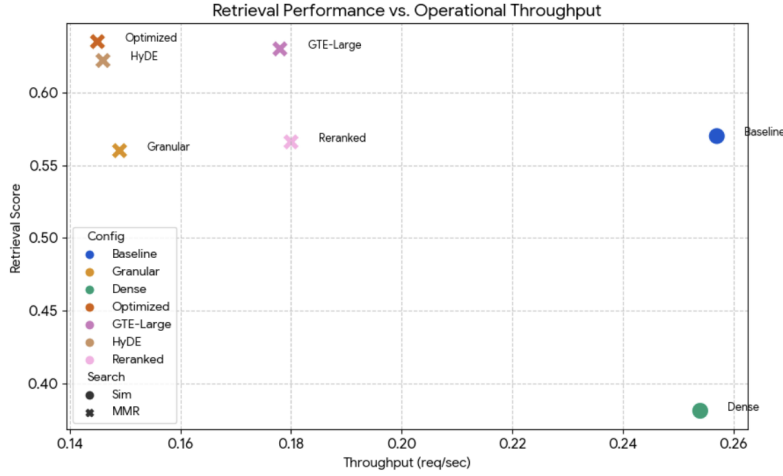


Figure 1: Comparative Analysis of Retrieval Scores vs. Operational Throughput

The chart underscores the transition from standard similarity-based retrieval to advanced techniques. It specifically highlights the efficacy of HyDE, which bridges the semantic gap between vague queries and technical documents to achieve the peak generation grounding score observed in our study.

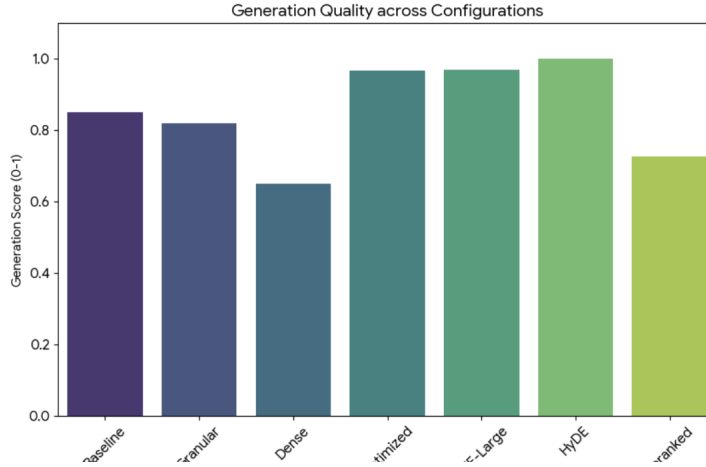


Figure 2: Impact of Advanced Retrieval Strategies on Generation Quality

The chart underscores the transition from standard similarity-based retrieval to advanced techniques. It specifically highlights the efficacy of HyDE, which bridges the semantic gap between vague queries and technical documents to achieve the peak generation grounding score observed in our study.

We also experimented with multiple embedding models including BAAI/bge-small-en-v1.5, BAAI/bge-large-en-v1.5, and thenlper/gte-large. Embedding models had a major impact on retrieval metrics because embeddings directly determine semantic similarity inside the vector database. Surprisingly, the larger embedding models did not consistently outperform the smaller embedding model. BAAI/bge-small-en-v1.5 achieved the strongest retrieval score of approximately **0.635**, while larger models such as BAAI/bge-large-en-v1.5 sometimes produced lower retrieval

scores around **0.615**. This was somewhat surprising because larger embedding models are generally expected to capture semantic relationships more effectively. However, on our dataset, the smaller embedding model appeared to generalize better and retrieve more focused contextual matches.

We additionally experimented with reranking strategies using cross-encoder/ms-marco-MiniLM-L6-v2 and BAAI/bge-reranker-base. Reranking improved retrieval precision by reordering initially retrieved chunks using stronger semantic relevance models. In our experiments, reranking reduced noisy retrievals and improved retrieval consistency. However, stronger rerankers occasionally reduced generation quality and throughput due to the additional ranking stage and stricter filtering behavior. We also experimented with advanced retrieval techniques such as HyDE and score fusion. HyDE generated hypothetical answers before retrieval and improved generation quality substantially, achieving a generation score close to **1.0**. Score fusion improved ranking stability by combining retrieval and reranker scores but slightly reduced throughput because of additional computation overhead.

Among all the knobs we evaluated, chunk size and embedding models had the biggest impact on retrieval metrics, while HyDE and reranking had the strongest effect on generation quality. Chunking directly controlled semantic completeness and context fragmentation, making it the most influential retrieval parameter overall. One of the most surprising observations was that retrieval performance was more sensitive to chunking strategy than embedding model size. Another surprising result was that stronger rerankers sometimes improved retrieval relevance while simultaneously lowering generation quality due to over-filtering potentially useful context.

Our final pipeline used `chunk_size = 288`, `chunk_overlap = 72`, MMR retrieval, `k = 40`, BAAI/bge-small-en-v1.5 embeddings, and cross-encoder/ms-marco-MiniLM-L6-v2 reranking. This configuration consistently produced the best balance between retrieval quality, generation quality, and throughput.

If we had more time and computational budget, we would explore hybrid sparse and dense retrieval, adaptive chunking strategies, larger instruction-tuned rerankers, ColBERT-style retrieval, graph-based RAG pipelines, and multi-hop retrieval systems. One technique we are particularly interested in exploring is context stitching, where semantically related neighboring chunks are dynamically merged during retrieval to preserve broader contextual continuity and reduce context fragmentation issues observed in smaller chunk configurations. We believe this could further improve retrieval grounding and generation quality. Additionally, we would evaluate more advanced LLM judges and perform larger-scale hyperparameter sweeps across retrieval depth, reranker thresholds, query rewriting strategies, and advanced reranking pipelines to further optimize retrieval robustness, ranking stability, and generation grounding.

Table 2: System Performance Benchmarks

Configuration	Retrieval Score	Gen Score	F1 Score
Baseline (256/64 Sim)	0.570	0.850	0.500
Dense (512/32 Sim)	0.381	0.650	0.320
Optimized (288/72 MMR)	0.635	0.967	0.526
Advanced (HyDE + MMR)	0.622	1.000	0.507
GTE-Large (288/72 MMR)	0.630	0.970	0.519

5 AI Tools Statement

We used Claude and Codex for general assistance. Specifically, we used the LLMs to help navigate the project specifications. Additionally, we used the LLMs for debugging and implementation assistance for code, and word choice assistance for writing. LLMs were also used to help generate the golden QA evaluation set with semi-synthesis methods described in the lecture. Each golden set was reviewed and verified by us.